LAZY

VÀ TRUY VẤN

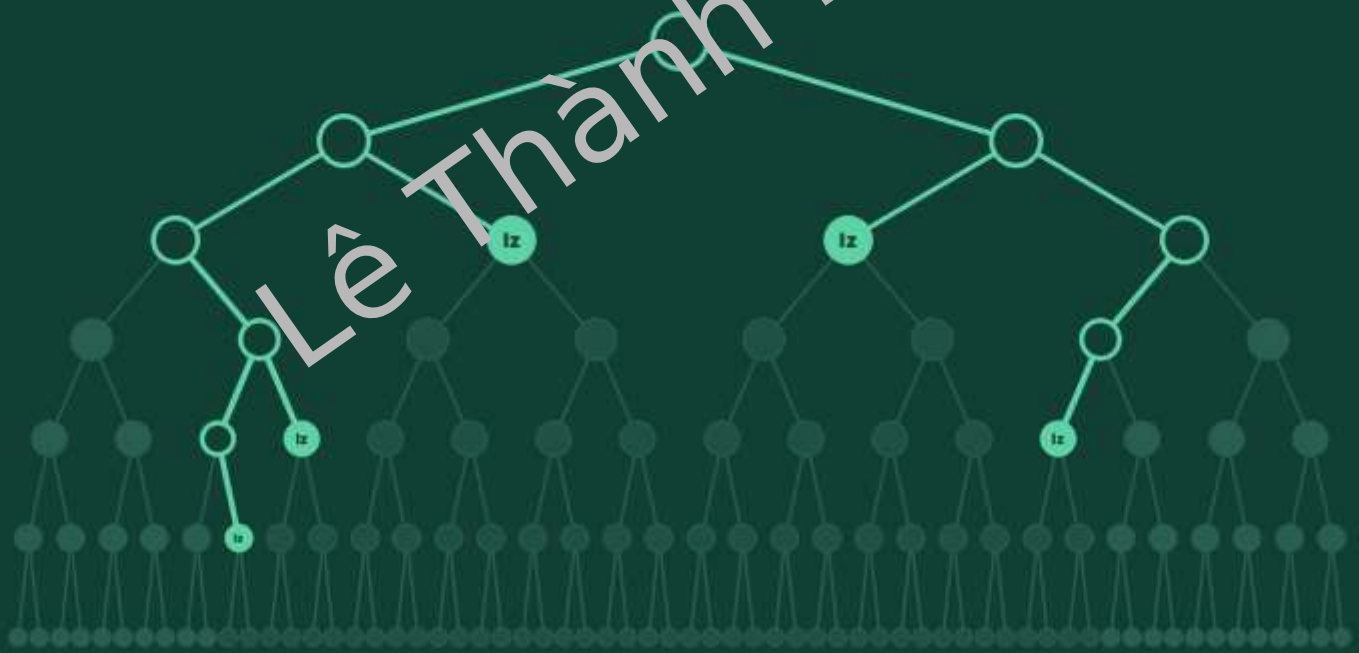
OFFLINE · ONLINE



Mảng hiệu · Cây phân đoạn · Fenwick · Lazy · Mo · DSU · Bền vững · Chia căn

sum[id] = sum[2id] + sum[2id + 1]
mỗi nút giữ thông tin một đoạn

update(11, 52)
64 lá, chỉ dừng ở 5 nút



Lê Thành Nam

a[l..r] += v → O(log n) nút + nhãn lazy

01	02	03	04	05
Truy vấn đoạn	Kĩ thuật lazy	Lazy nâng cao	Truy vấn offline	Online, kết hợp

TẬP 3 · HƯỚNG DẪN GIẢI

45 lời giải · từ cách ngây thơ đến tối ưu · bảng điểm thật

Fenwick · Mo · DSU · Chặt nhị phân song song · Bền vững

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho 48 bài tập của Tập 2. Mỗi lời giải không chỉ đưa ra chương trình đạt 100 điểm, mà còn chỉ ra **vì sao** cấu trúc dữ liệu ấy đúng (thông tin ở nút có ý nghĩa gì, nhãn hợp với nhau ra sao, vì sao tổng chi phí nhỏ), và **vì sao** những chương trình "trông hợp lí" khác lại sai hoặc chậm.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test chính thức với giới hạn 1 giây, 256 MB. Các con số trong **bảng điểm** cuối mỗi bài là điểm thực tế mà từng chương trình nhận được, không phải ước lượng. Nhiều bảng điểm có cả những chương trình **sai**: tràn số, lệch một chỉ số, quên đẩy nhãn, hợp nhãn sai thứ tự, dùng 0 làm dấu "không có nhãn", nén toạ độ mất khe hở, xử lí sai các sự kiện cùng toạ độ, chia cho 2 sau khi đã lấy dư, quên bỏ qua những lệnh "không làm gì", hoàn tác lệnh đổi giá trị bằng giá trị mới thay vì giá trị cũ, nén đường đi trong DSU cần hoàn tác, hoặc đúng nhưng quá chậm. Đó là những lỗi mà rất nhiều học sinh mắc phải, được giữ lại để em thấy chúng mất bao nhiêu điểm. Một số lỗi chỉ lộ ra ở vài test được thiết kế riêng: mục **Về bộ test** giải thích những test ấy được tạo ra như thế nào.

Lê Thành Nam

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
Ý tưởng chính	Cấu trúc dữ liệu hay kĩ thuật được dùng, kèm lập luận vì sao nó đúng và đủ nhanh.
Về bộ test	Những test được thiết kế để làm lộ các cách làm sai (nếu có).
Các cách chưa đúng	Từng chương trình sai hoặc chậm trong bảng điểm và lí do nó mất điểm.
Cách tối ưu	Chương trình hoàn chỉnh và độ phức tạp.
Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Lời khuyên: Đừng đọc lời giải như đọc truyện. Đọc mục đầu tiên, gấp sách và tự viết chương trình. Sau đó đọc "các cách chưa đúng": nếu chương trình của em trùng với một trong số đó, em đã biết mình mất bao nhiêu điểm và vì sao.

Cách tự đối chiếu với lời giải vét cạn

Ba tệp là đủ: chương trình của em (sol.cpp), chương trình vét cạn (trau.cpp, thường là duyệt đoạn trực tiếp) và một chương trình sinh test ngẫu nhiên nhỏ (gen.cpp, nhận một hạt giống từ dòng lệnh). Đoạn lệnh dưới đây chạy hàng trăm test và dừng lại ở test đầu tiên hai chương trình cho kết quả khác nhau.

‹› Đối chiếu tự động (chạy trên Linux hoặc Git Bash)

```
for i in $(seq 1 500); do
    ./gen $i > test.inp
    ./sol < test.inp > sol.out
    ./trau < test.inp > trau.out
    if ! cmp -s sol.out trau.out; then echo "Khác nhau ở test $i"; break; fi
done
```

Lê Thành Nam

Với các bài cây phân đoạn, hãy sinh test có **n rất nhỏ** (1 tới 8) nhưng **nhiều lệnh** (hàng chục), giá trị nhỏ (để có nhiều số bằng nhau), có cả giá trị 0 và giá trị âm, và các đoạn dài sát biên [1, n]: phần lớn lỗi lazy lộ ra ở đó chỉ sau vài chục test.

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. TRUY VẤN ĐOẠN CƠ BẢN **6**

Bài 1.1 TỔNG ĐOẠN 6
Bài 1.2 PHÂN PHÁT 9
Bài 1.3 PHỦ SÓNG 11
Bài 2.1 TỔNG SỬA 14
Bài 2.2 ĐOẠN CON 16
Bài 2.3 NGOẶC 19
Bài 3.1 NGHỊCH THẾ 22
Bài 3.2 CỘNG TỔNG 24
Bài 3.3 LƯỚI ĐIỂM 26

CHƯƠNG 2. KĨ THUẬT LAZY **28**

Bài 4.1 CAO MAX 28
Bài 4.2 ĐẶT PHÒNG 31
Bài 4.3 CẤP SỐ CỘNG 33
Bài 5.1 SƠN TƯỜNG 36
Bài 5.2 GÁN LẺ 39
Bài 5.3 NHÂN CỘNG 42
Bài 6.1 TỔNG BÌNH PHƯƠNG 45
Bài 6.2 KHÁCH SẠN 48
Bài 6.3 XOR ĐOẠN 51

CHƯƠNG 3. LAZY NÂNG CAO **54**

Bài 7.1 CHIẾU SÁNG 54
Bài 7.2 CÂY ĐỘNG 56
Bài 7.3 GHẾ NGỒI 59
Bài 8.1 ĐOẠN PHỦ 62
Bài 8.2 CHU VI 64
Bài 8.3 CỬA SỔ 67
Bài 9.1 TÌM VỊ TRÍ 70
Bài 9.2 CHIA LẤY 72
Bài 9.3 MOD ĐOẠN 75

CHƯƠNG 4. TRUY VẤN OFFLINE **78**

Bài 10.1 ĐẾM NHỎ 78
Bài 10.2 TỔNG KHÁC 81
Bài 10.3 MEX ĐOẠN 83

Lê Thành Nam

Bài 11.1 SỨC MẠNH....86
Bài 11.2 XOR CẶP....88
Bài 11.3 MO SỬA....91
Bài 12.1 CẮT ĐƯỜNG....94
Bài 12.2 TẢI ĐƯỜNG....96
Bài 12.3 ĐẢO NỐI....98
Bài 12.4 SƠN ĐÈ....100
Bài 12.5 PHÁ CẦU....103
Bài 12.6 THÊM XOÁ....107

CHƯƠNG 5. TRUY VẤN ONLINE VÀ KĨ THUẬT KẾT HỢP....111

Bài 13.1 ĐƯỜNG NGẬP....111
Bài 13.2 K NHỎ....114
Bài 13.3 SAO BĂNG....116
Bài 14.1 PHIÊN BẢN....119
Bài 14.2 ĐẾM NHỎ ONLINE....121
Bài 14.3 KHÁC ONLINE....123
Bài 15.1 KHỐI TỔNG....126
Bài 15.2 BÓNG NẢY....128
Bài 15.3 DỊCH VÒNG....131

TỔNG KẾT: 48 BÀI, MỘT BẢNG ÔN TẬP....134

Lê Thành Nam

CHƯƠNG 1

TRUY VẤN ĐOẠN CƠ BẢN

Lời giải của chín bài đều ngắn, nhưng mỗi bài có một chỗ dễ sai mà bảng điểm cho thấy rất rõ: cộng dồn bằng int (TỔNG ĐOẠN), trừ ở r thay vì r + 1 (PHÂN PHÁT), quên góc thứ tư của mảng hiệu hai chiều (PHỦ SÓNG), lưu tổng trên cây bằng int (TỔNG SỬA), ghép thiếu đoạn vắt qua điểm giữa (ĐOẠN CON), chỉ kiểm tra tổng ngoặc bằng 0 (NGOẶC), quên nén giá trị (NGHỊCH THẾ), sai hệ số trong cây Fenwick thứ hai (CỘNG TỔNG), và dùng một cây Fenwick cho mỗi hàng (LƯỚI ĐIỂM).

BÀI 1.1 TỔNG ĐOẠN

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TONGDOAN.INP	TONGDOAN.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1

Lê Thành Nam

1. Mảng cộng dồn

Dựng s[0] = 0, s[i] = s[i − 1] + a[i]; mỗi câu hỏi trả lời s[r] − s[l − 1] trong O(1). Điểm duy nhất cần suy nghĩ là **kiểu dữ liệu**: $|a_i| \le 10^9$ và $n \le 2 \cdot 10^5$ nên |s[i]| có thể tới $2 \cdot 10^{14}$, vượt xa phạm vi int (khoảng $2{,}1 \cdot 10^9$). Phải dùng long long cho mảng s.

2. Các cách chưa đúng

■ ĐIỂM BẾ TẮC

- loop.cpp cộng từng phần tử cho mỗi câu hỏi: O(n·q), đúng ở subtask 1 nhưng quá thời gian ở mọi test lớn.
- int.cpp dùng mảng cộng dồn kiểu int. Điều thú vị: chương trình này **không sai hết**. Khi tràn số, phép cộng int thực chất được tính theo modulo 2^{32} (trên mọi máy tính thông dụng), nên hiệu s[r] − s[l − 1] vẫn đúng theo modulo 2^{32}. Nếu đáp số thật nằm trong phạm vi int thì kết quả in ra vẫn đúng! Chương trình chỉ sai ở những câu hỏi có đáp số vượt phạm vi int, và mỗi test lớn có rất nhiều câu như vậy.

■ TRÀN SỐ CÓ THỂ "TRÔNG NHƯ ĐÚNG"

Hiện tượng trên giải thích vì sao lỗi tràn số rất khó phát hiện bằng test nhỏ: chương trình chạy đúng trên các ví dụ có đáp số nhỏ. Trong C++, tràn số với kiểu có dấu về lí thuyết là hành vi không xác định, nên đừng bao giờ dựa vào nó. Ví dụ thứ hai trong đề bài được đưa vào chính để nhắc điều này.

3. Cách tối ưu

TONGDOAN.CPP: Mảng cộng dồn long long

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TONGDOAN.INP", "r")) {
        freopen("TONGDOAN.INP", "r", stdin);
        freopen("TONGDOAN.OUT", "w", stdout);
    }
    int n, q;
    cin >> n >> q;
    vector<ll> s(n + 1, 0);          // s[i] = a[1] + ... + a[i], s[0] = 0
    for (int i = 1; i <= n; i++) {
        ll x;
        cin >> x;
        s[i] = s[i - 1] + x;
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << s[r] - s[l - 1] << '\n';   // tổng a[l..r]
    }
    return 0;
}
```

Độ phức tạp: O(n + q).

Lê Thành Nam

4. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (70)	Tổng
Cộng từng phần tử cho mỗi câu hỏi	30	14	**44**
Mảng cộng dồn kiểu int	20	7	**27**
Mảng cộng dồn long long	30	70	**100**

Lê Thành Nam

BÀI 1.2 PHÂN PHÁT

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
PHANPHAT.INP	PHANPHAT.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Bài 1

1. Sửa trước, hỏi sau

Mọi chuyến xe đều chạy xong trước mọi câu hỏi: đây đúng là tình huống "sửa trước, hỏi sau" của Bài 1. Ta làm ba bước:

- dựng mảng hiệu d[i] = a[i] − a[i − 1];
- mỗi chuyến xe (l, r, v): d[l] += v, d[r + 1] −= v;
- cộng dồn d để được dãy cuối cùng b, rồi cộng dồn b để được mảng s, mỗi câu hỏi trả lời s[y] − s[x − 1].

Chương trình mẫu gộp hai lần cộng dồn vào một vòng lặp: biến cuộn là b[i], mảng s là cộng dồn của b.

■ VÍ DỤ

a = (1, 2, 3, 4, 5), d = (1, 1, 1, 1, 1). Chuyến (1, 3, 10): d = (11, 1, 1, −9, 1). Chuyến (2, 5, 1): d = (11, 2, 1, −9, 1) và d[6] = −1. Cộng dồn: b = (11, 13, 14, 5, 6). Cộng dồn lần nữa: s = (11, 24, 38, 43, 49). Câu hỏi (4, 5): s[5] − s[3] = 11.

Giá trị lớn nhất: mỗi trạm nhận tối đa $2 \cdot 10^5 \cdot 10^6 = 2 \cdot 10^{11}$ chai, tổng một đoạn tới $4 \cdot 10^{16}$: dùng long long.

2. Các cách chưa đúng

■ ĐIỂM BẾ TẮC

- naive.cpp cộng v vào từng trạm cho mỗi chuyến xe: O(n·m). Đúng ở subtask 1, quá thời gian ở hai subtask sau.
- qloop.cpp dùng mảng hiệu cho các chuyến xe (O(1) mỗi chuyến) nhưng trả lời câu hỏi bằng cách cộng từng trạm: O(n) mỗi câu hỏi. Ở subtask 2, câu hỏi chỉ có x = y nên mỗi câu O(1), qua; ở subtask 3, quá thời gian. Đây là kiểu "làm được nửa bài": nhớ mảng hiệu nhưng quên rằng câu hỏi tổng đoạn cần thêm một lần cộng dồn.
- off.cpp viết d[r] −= v thay vì d[r + 1] −= v: trạm r không được nhận nước. Sai ở mọi test.

Lê Thành Nam

3. Cách tối ưu

PHANPHAT.CPP: Mảng hiệu rồi mảng cộng dồn

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("PHANPHAT.INP", "r")) {
        freopen("PHANPHAT.INP", "r", stdin);
        freopen("PHANPHAT.OUT", "w", stdout);
    }
    int n, m, q;
    cin >> n >> m >> q;
    vector<ll> a(n + 2, 0), d(n + 2, 0), s(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= n; i++) d[i] = a[i] - a[i - 1];   // mảng hiệu
    for (int j = 0; j < m; j++) {
        int l, r; ll v;
        cin >> l >> r >> v;
        d[l] += v;          // từ l trở đi tăng v
        d[r + 1] -= v;      // từ r + 1 trở đi trả lại như cũ
    }
    ll cur = 0;
    for (int i = 1; i <= n; i++) {
        cur += d[i];                // cur = a[i] sau mọi lần phát
        s[i] = s[i - 1] + cur;      // cộng dồn lần hai
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << s[r] - s[l - 1] << '\n';
    }
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: O(n + m + q).

4. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Cộng từng phần tử cho mỗi lần phát	30	0	0	**30**
Mảng hiệu, trừ nhầm tại r thay vì r + 1	0	0	0	**0**
Mảng hiệu, câu hỏi cộng từng phần tử	30	30	0	**60**
Mảng hiệu rồi mảng cộng dồn	30	30	40	**100**